

✓ Lab 6 – Ensemble Learning

Lab này khảo sát các mô hình học tổ hợp trên bài toán dự đoán sống sót của hành khách Titanic. Dữ liệu có cả biến số, biến phân loại và giá trị thiếu, nên phù hợp để kiểm tra toàn bộ quy trình: tiền xử lý, huấn luyện, đánh giá và diễn giải kết quả.

Trực phân tích chính là so sánh mô hình đơn với mô hình tổ hợp. Các mô hình đơn gồm Logistic Regression, KNN, SVM và Decision Tree. Các mô hình tổ hợp gồm Voting, Bagging, Pasting, Random Forest, Extra Trees và nhóm Boosting.

Điểm quan trọng không chỉ là mô hình nào có điểm cao nhất, mà là vì sao một nhóm mô hình ổn định hơn nhóm khác. Với Titanic, các tín hiệu như giới tính, hạng vé, tuổi, giá vé, quy mô gia đình và danh xưng đều có thể ảnh hưởng đến khả năng sống sót.

```
import os
import time
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn

from sklearn.model_selection import train_test_split, cross_validate, StratifiedKFold
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.svm import SVC
from sklearn.dummy import DummyClassifier

from sklearn.ensemble import (
    VotingClassifier,
    BaggingClassifier,
    RandomForestClassifier,
    ExtraTreesClassifier,
    AdaBoostClassifier,
    GradientBoostingClassifier,
    HistGradientBoostingClassifier
)

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report,
    roc_curve
)

warnings.filterwarnings("ignore")

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (9, 6)

print("Môi trường sẵn sàng.")
print("NumPy:", np.__version__)
print("Pandas:", pd.__version__)
print("Scikit-learn:", sklearn.__version__)
```

Môi trường sẵn sàng.
NumPy: 2.0.2
Pandas: 2.2.2
Scikit-learn: 1.6.1

1. Nạp dữ liệu Titanic

Notebook ưu tiên đọc `train.csv` nếu file đã được upload. Khi không có file cục bộ, dữ liệu Titanic được lấy từ một nguồn CSV công khai. Trường hợp môi trường không có Internet, một bộ dữ liệu mô phỏng nhỏ sẽ được tạo để notebook vẫn chạy được, nhưng phần phân tích chính vẫn dựa trên Titanic chuẩn khi nguồn dữ liệu khả dụng.

```
def make_fallback_titanic(n=891, random_state=RANDOM_STATE):
    rng = np.random.default_rng(random_state)
    sex = rng.choice(["male", "female"], size=n, p=[0.65, 0.35])
    pclass = rng.choice([1, 2, 3], size=n, p=[0.24, 0.21, 0.55])
    age = rng.normal(30, 14, size=n).clip(0.4, 80)
    sibsp = rng.choice([0, 1, 2, 3, 4], size=n, p=[0.68, 0.22, 0.06, 0.03, 0.01])
    parch = rng.choice([0, 1, 2, 3], size=n, p=[0.75, 0.15, 0.08, 0.02])
    fare = rng.lognormal(mean=2.8, sigma=0.9, size=n)
    embarked = rng.choice(["S", "C", "Q"], size=n, p=[0.72, 0.19, 0.09])
    title = np.where(sex == "female", rng.choice(["Miss", "Mrs"], size=n), rng.choice(["Mr", "Master"], size=n, p=[0.65, 0.35]))

    score = (
        -1.2
        + 1.9 * (sex == "female")
        + 0.8 * (pclass == 1)
        + 0.35 * (pclass == 2)
        - 0.015 * np.nan_to_num(age, nan=30)
        + 0.25 * (embarked == "C")
        + 0.25 * (title == "Master")
        - 0.08 * (sibsp + parch)
    )
    prob = 1 / (1 + np.exp(-score))
    survived = (rng.random(n) < prob).astype(int)

    df_fb = pd.DataFrame({
        "PassengerId": np.arange(1, n + 1),
        "Survived": survived,
        "Pclass": pclass,
        "Name": [f"Passenger, {t}. Name" for t in title],
        "Sex": sex,
        "Age": age,
        "SibSp": sibsp,
        "Parch": parch,
        "Ticket": [f"T{i}" for i in range(n)],
        "Fare": fare,
        "Cabin": np.nan,
        "Embarked": embarked,
    })

    missing_age_idx = rng.choice(n, size=int(0.18 * n), replace=False)
    df_fb.loc[missing_age_idx, "Age"] = np.nan
    return df_fb

possible_paths = [
    "/content/train.csv",
    "/content/titanic/train.csv",
    "train.csv",
    "/mnt/data/train.csv"
]

data_path = None
for path in possible_paths:
    if os.path.exists(path):
        data_path = path
        break

if data_path is not None:
    df_raw = pd.read_csv(data_path)
    source_name = f"Titanic train.csv từ file: {data_path}"
else:
    url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
    try:
        df_raw = pd.read_csv(url)
        source_name = "Titanic CSV công khai từ datasciencedojo/datasets"
    except Exception:
        df_raw = make_fallback_titanic()
        source_name = "Fallback Titanic-like dataset, dùng khi môi trường không có Internet"
```

```
print("Nguồn dữ liệu:", source_name)
print("Kích thước dữ liệu:", df_raw.shape)
display(df_raw.head())
```

Nguồn dữ liệu: Titanic CSV công khai từ datasciencedojo/datasets
 Kích thước dữ liệu: (891, 12)

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C	
2	3	1	3	Heikinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S	

2. Kiểm tra cấu trúc dữ liệu

Biến mục tiêu là `Survived`, trong đó `1` biểu diễn hành khách sống sót và `0` biểu diễn không sống sót. Các biến như `Sex`, `Pclass`, `Age`, `Fare`, `SibSp`, `Parch` và `Embarked` được giữ lại vì có ý nghĩa trực tiếp trong bài toán. Một số cột như `Name` sẽ được khai thác sau để tạo `Title`.

Các giá trị thiếu được kiểm tra trước khi huấn luyện. Việc xử lý thiếu không làm trực tiếp trên toàn bộ dữ liệu mà được đặt trong `Pipeline`, nhờ đó dữ liệu test không bị rò rỉ thông tin vào quá trình học.

```
df = df_raw.copy()

if "survived" in df.columns and "Survived" not in df.columns:
    df = df.rename(columns={
        "survived": "Survived",
        "pclass": "Pclass",
        "sex": "Sex",
        "age": "Age",
        "sibsp": "SibSp",
        "parch": "Parch",
        "fare": "Fare",
        "embarked": "Embarked"
    })

required_cols = ["Survived", "Pclass", "Sex", "Age", "SibSp", "Parch", "Fare", "Embarked"]
available_required = [c for c in required_cols if c in df.columns]

display(df[available_required].head())
print("Thông tin kiểu dữ liệu:")
df[available_required].info()

missing_table = pd.DataFrame({
    "Số giá trị thiếu": df[available_required].isna().sum(),
    "Tỷ lệ thiếu (%)": df[available_required].isna().mean() * 100
}).sort_values("Số giá trị thiếu", ascending=False)

display(missing_table)

print("Nhận xét:")
print(f"Dữ liệu có {df.shape[0]} mẫu và {df.shape[1]} cột.")
print(f"Cột thiếu nhiều nhất là {missing_table.index[0]} với {int(missing_table.iloc[0]['Số giá trị thiếu'])} giá trị")
print("Age thiếu khá nhiều nên cần impute bằng median trong pipeline. Embarked nếu thiếu ít có thể impute bằng giá trị")
print("Cách xử lý này giữ quy trình train/test sạch hơn so với điền thiếu trước khi chia dữ liệu.")
```

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	0	3	male	22.0	1	0	7.2500	S
1	1	1	female	38.0	1	0	71.2833	C
2	1	3	female	26.0	0	0	7.9250	S
3	1	1	female	35.0	1	0	53.1000	S
4	0	3	male	35.0	0	0	8.0500	S

Thông tin kiểu dữ liệu:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 891 entries, 0 to 890

Data columns (total 8 columns):

#	Column	Non-Null Count	Dtype
0	Survived	891 non-null	int64
1	Pclass	891 non-null	int64
2	Sex	891 non-null	object
3	Age	714 non-null	float64
4	SibSp	891 non-null	int64
5	Parch	891 non-null	int64
6	Fare	891 non-null	float64
7	Embarked	889 non-null	object

dtypes: float64(2), int64(4), object(2)

memory usage: 55.8+ KB

Số giá trị thiếu Tỷ lệ thiếu (%) 

Age	177	19.865320
Embarked	2	0.224467
Pclass	0	0.000000
Survived	0	0.000000
Sex	0	0.000000
SibSp	0	0.000000
Parch	0	0.000000
Fare	0	0.000000

Nhận xét:

Dữ liệu có 891 mẫu và 12 cột.

Cột thiếu nhiều nhất là Age với 177 giá trị thiếu.

Age thiếu khá nhiều nên cần impute bằng median trong pipeline. Embarked nếu thiếu ít có thể impute bằng giá trị phổ biến.

Cách xử lý này giữ quy trình train/test sạch hơn so với điền thiếu trước khi chia dữ liệu.

Các bước tiếp theo:

[Tạo mã bằng missing_table](#)

[New interactive sheet](#)

3. Phân bố nhãn

Tỷ lệ sống sót là mốc đầu tiên để hiểu độ khó của bài toán. Trong Titanic, số hành khách không sống sót lớn hơn số hành khách sống sót. Dữ liệu không mất cân bằng quá mạnh, nhưng nếu chỉ nhìn Accuracy vẫn có thể bỏ qua việc mô hình dự đoán lớp sống sót tốt hay không.

```
target_col = "Survived"

class_table = pd.DataFrame({
    "Số mẫu": df[target_col].value_counts().sort_index(),
    "Tỷ lệ (%)": df[target_col].value_counts(normalize=True).sort_index() * 100
})
class_table.index = ["Không sống sót (0)", "Sống sót (1)"]

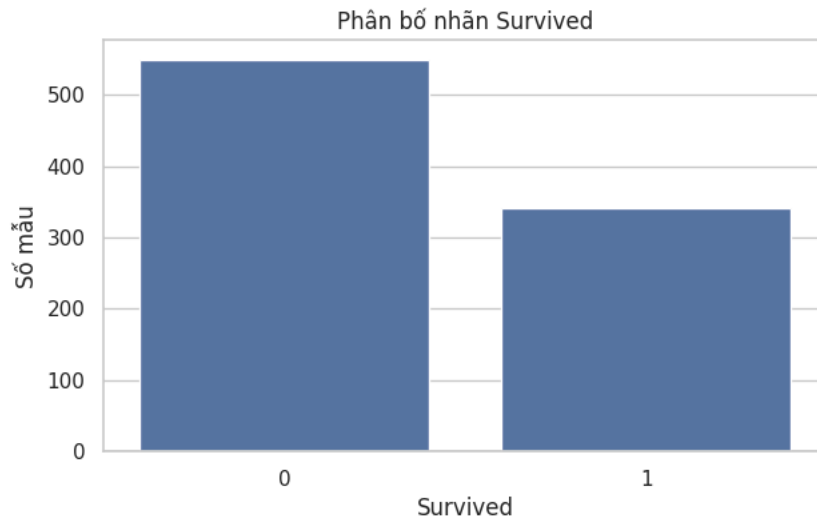
display(class_table)

plt.figure(figsize=(7, 4))
sns.countplot(data=df, x=target_col)
plt.title("Phân bố nhãn Survived")
plt.xlabel("Survived")
plt.ylabel("Số mẫu")
plt.show()

survival_rate = df[target_col].mean()
majority_rate = df[target_col].value_counts(normalize=True).max()
```

```
print("Nhận xét:")
print(f"Tỷ lệ sống sót chung = {survival_rate * 100:.2f}%.")
print(f"Nếu luôn đoán lớp phổ biến nhất, Accuracy nên khoảng {majority_rate:.4f}.")
print("Do đó các mô hình phía sau cần vượt baseline này và đồng thời cải thiện F1/Recall của lớp sống sót.")
```

	Số mẫu	Tỷ lệ (%)	
Không sống sót (0)	549	61.616162	
Sống sót (1)	342	38.383838	



Nhận xét:
Tỷ lệ sống sót chung = 38.38%.
Nếu luôn đoán lớp phổ biến nhất, Accuracy nên khoảng 0.6162.
Do đó các mô hình phía sau cần vượt baseline này và đồng thời cải thiện F1/Recall của lớp sống sót.

Các bước tiếp theo: [Tạo mã bằng class_table](#) [New interactive sheet](#)

4. Tín hiệu dự đoán từ các biến chính

Một số biến có quan hệ rất rõ với khả năng sống sót. Việc tính tỷ lệ sống sót theo nhóm giúp kiểm tra trước liệu mô hình có tín hiệu để học hay không. Nếu tỷ lệ giữa các nhóm chênh lệch mạnh, biến đó thường có giá trị phân loại cao.

```
def group_survival_rate(data, col):
    table = data.groupby(col)[target_col].agg(["count", "mean"]).reset_index()
    table = table.rename(columns={"count": "Số mẫu", "mean": "Tỷ lệ sống sót"})
    return table.sort_values("Tỷ lệ sống sót", ascending=False)

signal_tables = {}
for col in ["Sex", "Pclass", "Embarked"]:
    if col in df.columns:
        table = group_survival_rate(df, col)
        signal_tables[col] = table
        display(table)

    plt.figure(figsize=(7, 4))
    sns.barplot(data=table, x=col, y="Tỷ lệ sống sót")
    plt.ylim(0, 1)
    plt.title(f"Tỷ lệ sống sót theo {col}")
    plt.ylabel("Tỷ lệ sống sót")
    plt.show()

print("Nhận xét:")
if "Sex" in signal_tables:
    female_rate = signal_tables["Sex"].set_index("Sex").loc["female", "Tỷ lệ sống sót"]
    male_rate = signal_tables["Sex"].set_index("Sex").loc["male", "Tỷ lệ sống sót"]
    print(f"Tỷ lệ sống sót của nữ là {female_rate:.3f}, cao hơn rõ so với nam là {male_rate:.3f}.")
if "Pclass" in signal_tables:
    pclass_table = signal_tables["Pclass"].set_index("Pclass")
    print(f"Hạng 1 có tỷ lệ sống sót {pclass_table.loc[1, 'Tỷ lệ sống sót']:.3f}, trong khi hạng 3 chỉ khoảng {pclass_table.loc[3, 'Tỷ lệ sống sót']:.3f}.")
print("Các chênh lệch này giải thích vì sao Sex, Pclass và các biến liên quan đến địa vị xã hội thường xuất hiện mạnh
```


	Sex	Số mẫu	Tỷ lệ sống sót
0	female	314	0.742038
1	male	577	0.188908

Tỷ lệ sống sót theo Sex

Các bước tiếp theo:

Tạo mã bằng table

New interactive sheet

Tạo mã bằng table

New interactive sheet

Tạo mã bằng table

New interactive sheet

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

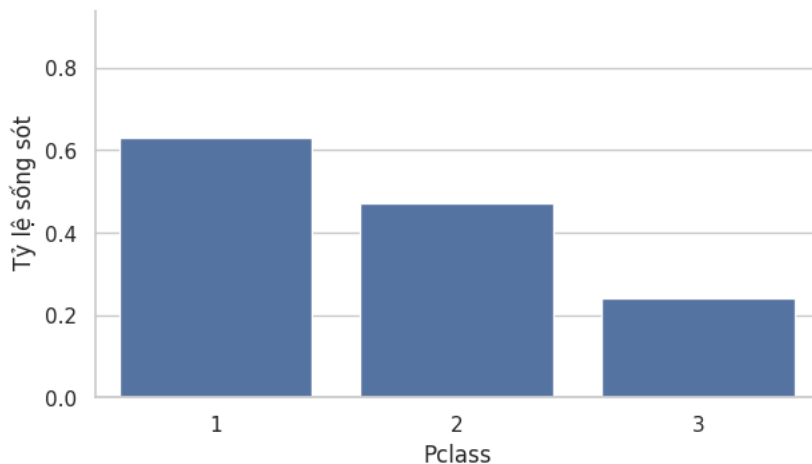
if "Age" in df.columns:
    sns.histplot(data=df, x="Age", hue=target_col, bins=30, kde=True, stat="density", common_norm=False, ax=axes[0])
    axes[0].set_title("Phân phối Age theo Survived")
    axes[0].set_xlabel("Age")

if "Fare" in df.columns:
    sns.histplot(data=df, x="Fare", hue=target_col, bins=35, kde=True, stat="density", common_norm=False, ax=axes[1])
    axes[1].set_title("Phân phối Fare theo Survived")
    axes[1].set_xlabel("Fare")
    axes[1].set_xlim(0, df["Fare"].quantile(0.98))

plt.tight_layout()
plt.show()

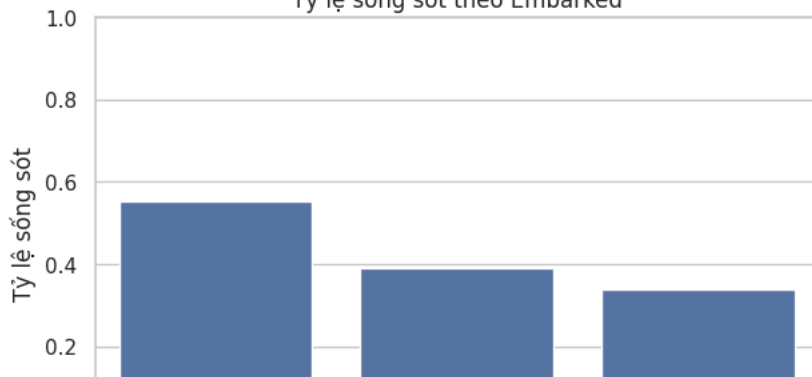
numeric_cols_check = [c for c in ["Age", "SibSp", "Parch", "Fare", "Pclass", "Survived"] if c in df.columns]
display(df[numeric_cols_check].describe().T)

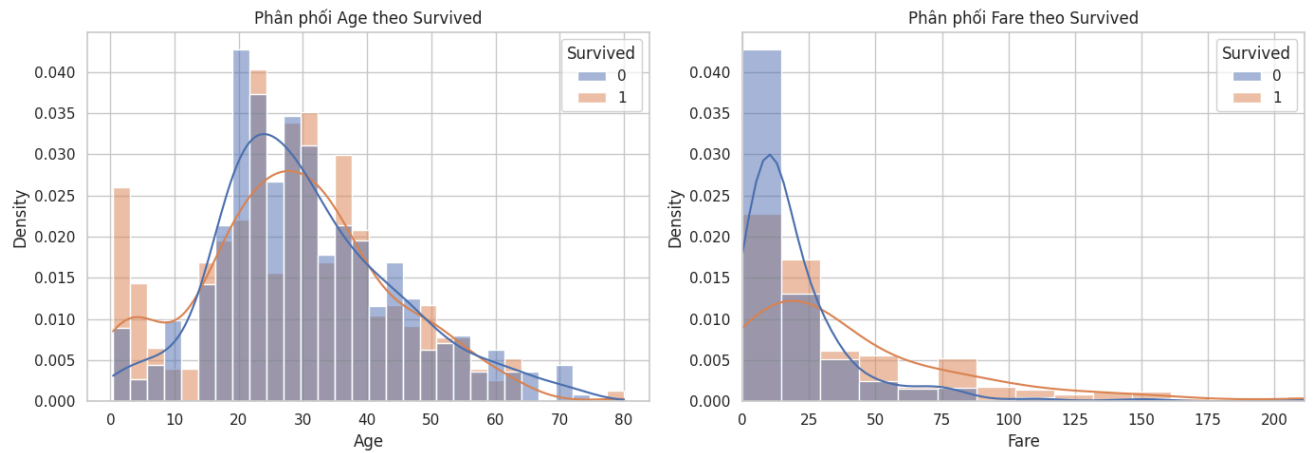
print("Nhận xét:")
print("Age tập trung nhiều quanh nhóm thanh niên và trung niên, nhưng vẫn có trẻ em và người lớn tuổi.")
print("Fare lệch phải mạnh: phần lớn hành khách trả giá vé thấp, trong khi một số ít có giá vé rất cao.")
print("Với Logistic Regression, KNN và SVM, scaling giúp các biến như Fare không lấn át các biến khác. Với mô hình c
```



	Embarked	Số mẫu	Tỷ lệ sống sót
0	C	168	0.553571
1	Q	77	0.389610
2	S	644	0.336957

Tỷ lệ sống sót theo Embarked





	count	mean	std	min	25%	50%	75%	max
Age	714.0	29.699118	14.526497	0.42	20.1250	28.0000	38.0	80.0000
SibSp	891.0	0.523008	1.102743	0.00	0.0000	0.0000	1.0	8.0000
Parch	891.0	0.381594	0.806057	0.00	0.0000	0.0000	0.0	6.0000
Fare	891.0	32.204208	49.693429	0.00	7.9104	14.4542	31.0	512.3292
Pclass	891.0	2.308642	0.836071	1.00	2.0000	3.0000	3.0	3.0000
Survived	891.0	0.383838	0.486592	0.00	0.0000	0.0000	1.0	1.0000

Nhận xét:

Age tập trung nhiều quanh nhóm thanh niên và trung niên, nhưng vẫn có trẻ em và người lớn tuổi.

Fare lệch phải mạnh: phần lớn hành khách trả giá vé thấp, trong khi một số ít có giá vé rất cao.

Với Logistic Regression, KNN và SVM, scaling giúp các biến như Fare không lấn át các biến khác. Với mô hình cây, scal

5. Tạo thêm biến từ cấu trúc gia đình và danh xưng

Hai biến `SibSp` và `Parch` được gộp thành `FamilySize` để biểu diễn quy mô nhóm đi cùng. Biến `IsAlone` cho biết hành khách đi một mình hay không. Từ `Name`, danh xưng như `Mr`, `Mrs`, `Miss`, `Master` được tách thành `Title` vì nó chứa thông tin gián tiếp về giới tính, độ tuổi và vị thế xã hội.

```
df_model = df.copy()

if "SibSp" in df_model.columns and "Parch" in df_model.columns:
    df_model["FamilySize"] = df_model["SibSp"] + df_model["Parch"] + 1
    df_model["IsAlone"] = (df_model["FamilySize"] == 1).astype(int)

if "Name" in df_model.columns:
    df_model["Title"] = df_model["Name"].str.extract(r"\s*([^\s.]+)\.", expand=False)
    common_titles = ["Mr", "Miss", "Mrs", "Master"]
    df_model["Title"] = df_model["Title"].where(df_model["Title"].isin(common_titles), "Other")

feature_candidates = [
    "Pclass", "Sex", "Age", "SibSp", "Parch", "Fare", "Embarked",
    "FamilySize", "IsAlone", "Title"
]
features = [c for c in feature_candidates if c in df_model.columns]
model_data = df_model[features + [target_col]].copy()

display(model_data.head())
display(model_data.isna().sum().to_frame("Số giá trị thiếu"))

if "FamilySize" in model_data.columns:
    family_table = group_survival_rate(model_data, "FamilySize")
    display(family_table.head(10))

plt.figure(figsize=(8, 4))
```



```
sns.barplot(data=family_table, x="FamilySize", y="Tỷ lệ sống sót")
plt.ylim(0, 1)
plt.title("Tỷ lệ sống sót theo FamilySize")
plt.xlabel("FamilySize")
plt.ylabel("Tỷ lệ sống sót")
plt.show()

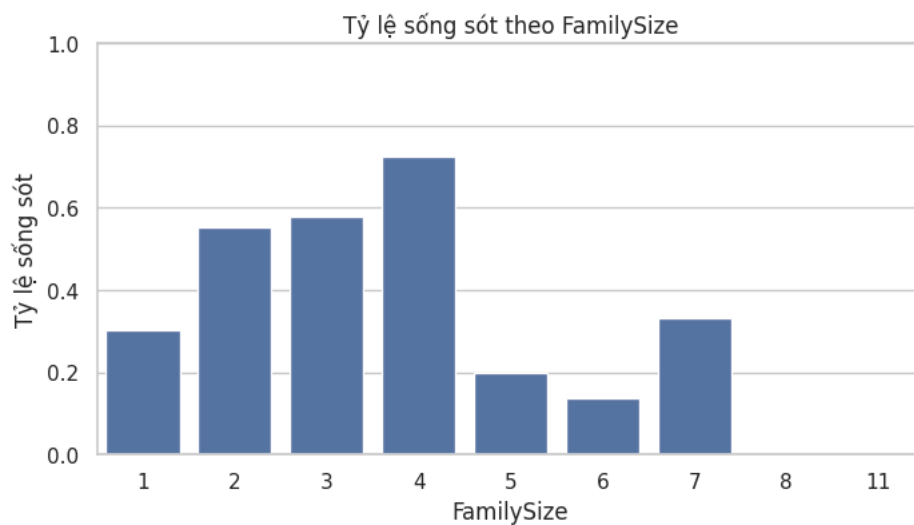
print("Nhận xét:")
print("Các nhóm gia đình nhỏ có tỷ lệ sống sót cao hơn nhóm đi một mình hoặc nhóm quá đông trong kết quả hiện tại.")
print("Title giúp rút gọn thông tin trong Name thành nhóm xã hội dễ dùng cho mô hình, đặc biệt là nhóm Mr, Mrs, Miss")
print("Các biến mới này không làm thay đổi bản chất dữ liệu, nhưng giúp mô hình có thêm cách biểu diễn thông tin sẵn
```

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	FamilySize	IsAlone	Title	Survived
0	3	male	22.0	1	0	7.2500	S	2	0	Mr	0
1	1	female	38.0	1	0	71.2833	C	2	0	Mrs	1
2	3	female	26.0	0	0	7.9250	S	1	1	Miss	1
3	1	female	35.0	1	0	53.1000	S	2	0	Mrs	1
4	3	male	35.0	0	0	8.0500	S	1	1	Mr	0

Số giá trị thiếu

Pclass	0
Sex	0
Age	177
SibSp	0
Parch	0
Fare	0
Embarked	2
FamilySize	0
IsAlone	0
Title	0
Survived	0

FamilySize	Số mẫu	Tỷ lệ sống sót
3	4	0.724138
2	3	0.578431
1	2	0.552795
6	7	0.333333
0	1	0.303538
4	5	0.200000
5	6	0.136364
7	8	0.000000
8	11	0.000000



Nhận xét:

Các nhóm gia đình nhỏ có tỷ lệ sống sót cao hơn nhóm đi một mình hoặc nhóm quá đông trong kết quả hiện tại.

Title giúp rút gọn thông tin trong Name thành nhóm xã hội dễ dùng cho mô hình, đặc biệt là nhóm Mr, Mrs, Miss và Ma:

Các biến mới này không làm thay đổi bản chất dữ liệu, nhưng giúp mô hình có thêm cách biểu diễn thông tin sẵn có.

6. Pipeline tiền xử lý

Dữ liệu được chia train/test bằng `stratify` để tỷ lệ sống sót ở hai tập gần nhau. Toàn bộ bước điển thiếu, chuẩn hóa và one-hot encoding được đặt trong `ColumnTransformer`. Cách này giúp mọi mô hình dùng cùng một đầu vào đã xử lý, đồng thời tránh xử lý dữ liệu test trước khi train.

```
X = model_data.drop(columns=[target_col])
y = model_data[target_col].astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    stratify=y,
    random_state=RANDOM_STATE
)

numeric_features = X.select_dtypes(include=["int64", "float64"]).columns.tolist()
categorical_features = X.select_dtypes(include=["object", "category", "bool"]).columns.tolist()

try:
    onehot = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
except TypeError:
    onehot = OneHotEncoder(handle_unknown="ignore", sparse=False)

numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", onehot)
])

preprocess = ColumnTransformer([
    ("num", numeric_pipeline, numeric_features),
    ("cat", categorical_pipeline, categorical_features)
])

print("Số dòng train:", X_train.shape[0])
print("Số dòng test:", X_test.shape[0])
print("Numeric features:", numeric_features)
print("Categorical features:", categorical_features)

split_table = pd.DataFrame({
    "Train": y_train.value_counts().sort_index(),
    "Test": y_test.value_counts().sort_index()
})
split_table.index = ["Không sống sót (0)", "Sống sót (1)"]
display(split_table)

print("Nhận xét:")
print(f"Tỷ lệ sống sót ở train = {y_train.mean() * 100:.2f}%.")
print(f"Tỷ lệ sống sót ở test = {y_test.mean() * 100:.2f}%.")
print("Hai tỷ lệ gần nhau, vì vậy tập test giữ được cấu trúc nhân của dữ liệu gốc.")
```

Số dòng train: 668
Số dòng test: 223
Numeric features: ['Pclass', 'Age', 'SibSp', 'Parch', 'Fare', 'FamilySize', 'IsAlone']
Categorical features: ['Sex', 'Embarked', 'Title']

	Train	Test	
Không sống sót (0)	412	137	
Sống sót (1)	256	86	

Nhận xét:
Tỷ lệ sống sót ở train = 38.32%.
Tỷ lệ sống sót ở test = 38.57%.
Hai tỷ lệ gần nhau, vì vậy tập test giữ được cấu trúc nhân của dữ liệu gốc.

Các bước tiếp theo: [Tạo mã bằng split_table](#) [New interactive sheet](#)

7. Baseline

Baseline đoán toàn bộ mẫu theo lớp phổ biến nhất. Đây không phải mô hình tốt, nhưng là mốc cần có. Một mô hình học máy chỉ có ý nghĩa nếu vượt baseline và cải thiện được các chỉ số liên quan đến lớp sống sót.

```
baseline = Pipeline([
    ("preprocess", preprocess),
    ("model", DummyClassifier(strategy="most_frequent"))
])

baseline.fit(X_train, y_train)
baseline_pred = baseline.predict(X_test)

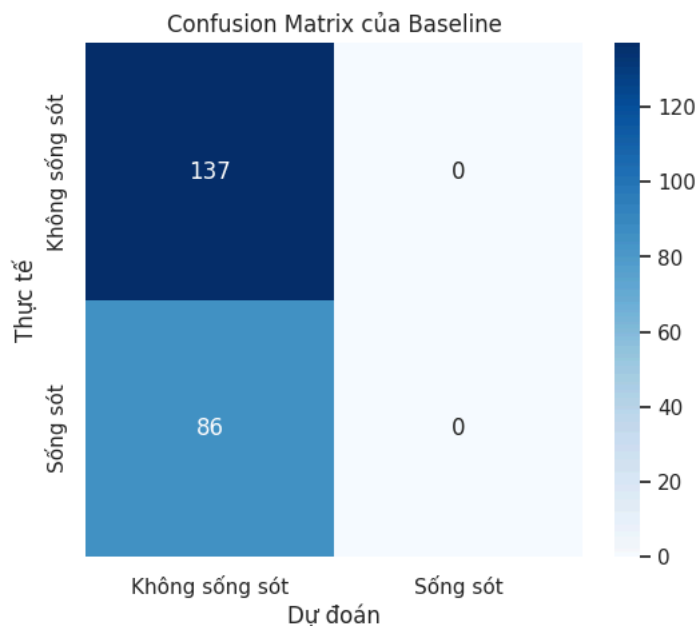
baseline_df = pd.DataFrame({
    "Mô hình": ["Baseline"],
    "Accuracy": [accuracy_score(y_test, baseline_pred)],
    "Precision": [precision_score(y_test, baseline_pred, zero_division=0)],
    "Recall": [recall_score(y_test, baseline_pred, zero_division=0)],
    "F1": [f1_score(y_test, baseline_pred, zero_division=0)]
})

display(baseline_df)

cm_base = confusion_matrix(y_test, baseline_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(
    cm_base,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Không sống sót", "Sống sót"],
    yticklabels=["Không sống sót", "Sống sót"]
)
plt.title("Confusion Matrix của Baseline")
plt.xlabel("Dự đoán")
plt.ylabel("Thực tế")
plt.show()

print("Nhận xét:")
print(f"Baseline đạt Accuracy = {baseline_df.loc[0, 'Accuracy']:.4f}.")
print("Precision, Recall và F1 của lớp sống sót bằng 0 vì mô hình không dự đoán hành khách nào sống sót.")
print("Kết quả này cho thấy Accuracy nên chưa đủ để đánh giá bài toán phân loại hai lớp.")
```

	Mô hình	Accuracy	Precision	Recall	F1	
0	Baseline	0.61435	0.0	0.0	0.0	



Nhận xét:
Baseline đạt Accuracy = 0.6143.
Precision, Recall và F1 của lớp sống sót bằng 0 vì mô hình không dự đoán hành khách nào sống sót.
Kết quả này cho thấy Accuracy nên chưa đủ để đánh giá bài toán phân loại hai lớp.

8. Mô hình đơn

Các mô hình đơn được dùng làm nền so sánh trước khi xét ensemble. Logistic Regression kiểm tra ranh giới tuyến tính sau tiền xử lý. KNN dựa vào khoảng cách giữa mẫu. SVM RBF tạo ranh giới phi tuyến. Decision Tree tạo các luật phân tách để giải thích nhưng có thể nhạy với dữ liệu.

```
def make_pipeline(model):
    return Pipeline([
        ("preprocess", preprocess),
        ("model", model)
    ])

base_models = {
    "Logistic Regression": make_pipeline(LogisticRegression(max_iter=2000, random_state=RANDOM_STATE)),
    "KNN": make_pipeline(KNeighborsClassifier(n_neighbors=9)),
    "Decision Tree": make_pipeline(DecisionTreeClassifier(max_depth=4, min_samples_leaf=8, random_state=RANDOM_STATE)),
    "SVM RBF": make_pipeline(SVC(kernel="rbf", C=1.5, gamma="scale", probability=True, random_state=RANDOM_STATE))
}

single_rows = []
for name, model in base_models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    single_rows.append({
        "Mô hình": name,
        "Accuracy": accuracy_score(y_test, pred),
        "Precision": precision_score(y_test, pred, zero_division=0),
        "Recall": recall_score(y_test, pred, zero_division=0),
        "F1": f1_score(y_test, pred, zero_division=0)
    })

single_df = pd.DataFrame(single_rows).sort_values("F1", ascending=False)
display(single_df)

best_single = single_df.iloc[0]
print("Nhận xét:")
print(f"Trong nhóm mô hình đơn, {best_single['Mô hình']} có F1 cao nhất = {best_single['F1']:.4f}.")
print("Logistic Regression và Decision Tree thường là hai mô quan trọng: một mô hình tuyến tính dễ ổn định và một mô hình phi tuyến")
print("Các mô hình đơn tạo nên đề đánh giá xem ensemble có thật sự cải thiện hay chỉ làm mô hình phức tạp hơn.")
```

	Mô hình	Accuracy	Precision	Recall	F1	
0	Logistic Regression	0.825112	0.783133	0.755814	0.769231	
2	Decision Tree	0.820628	0.767442	0.767442	0.767442	
3	SVM RBF	0.807175	0.786667	0.686047	0.732919	
1	KNN	0.780269	0.746667	0.651163	0.695652	

Nhận xét:

Trong nhóm mô hình đơn, Logistic Regression có F1 cao nhất = 0.7692.

Logistic Regression và Decision Tree thường là hai mô quan trọng: một mô hình tuyến tính dễ ổn định và một mô hình phi tuyến

Các mô hình đơn tạo nên đề đánh giá xem ensemble có thật sự cải thiện hay chỉ làm mô hình phức tạp hơn.

Các bước tiếp theo: [Tạo mã bằng single_df](#) [New interactive sheet](#)

9. Voting

Voting kết hợp nhiều mô hình khác loại. Hard Voting lấy nhãn theo đa số phiếu. Soft Voting lấy trung bình xác suất dự đoán, nên thường tận dụng được mức độ tự tin của từng mô hình. Với Titanic, Voting có ý nghĩa vì các mô hình nền học theo cơ chế khác nhau.

```
voting_estimators = [
    ("lr", base_models["Logistic Regression"]),
    ("knn", base_models["KNN"]),
    ("tree", base_models["Decision Tree"]),
    ("svm", base_models["SVM RBF"])
]

voting_hard = VotingClassifier(
    estimators=voting_estimators,
    voting="hard",
    n_jobs=-1
```

```

)

voting_soft = VotingClassifier(
    estimators=voting_estimators,
    voting="soft",
    n_jobs=-1
)

print("Đã khởi tạo Voting Hard và Voting Soft từ bốn mô hình nên.")

```

Đã khởi tạo Voting Hard và Voting Soft từ bốn mô hình nên.

10. Bagging và Pasting

Bagging và Pasting đều huấn luyện nhiều cây quyết định trên các tập con khác nhau. Bagging lấy mẫu có hoàn lại, còn Pasting lấy mẫu không hoàn lại. Với Bagging, OOB score có thể xem như một đánh giá nội bộ vì mỗi cây được kiểm tra trên những mẫu không được chọn vào bootstrap của chính cây đó.

```

def create_bagging_classifier(bootstrap=True, oob_score=False):
    base_tree = DecisionTreeClassifier(
        max_depth=None,
        min_samples_leaf=4,
        random_state=RANDOM_STATE
    )

    try:
        return BaggingClassifier(
            estimator=base_tree,
            n_estimators=180,
            max_samples=0.75,
            bootstrap=bootstrap,
            oob_score=oob_score,
            random_state=RANDOM_STATE,
            n_jobs=-1
        )
    except TypeError:
        return BaggingClassifier(
            base_estimator=base_tree,
            n_estimators=180,
            max_samples=0.75,
            bootstrap=bootstrap,
            oob_score=oob_score,
            random_state=RANDOM_STATE,
            n_jobs=-1
        )

bagging_model = make_pipeline(create_bagging_classifier(bootstrap=True, oob_score=True))
pasting_model = make_pipeline(create_bagging_classifier(bootstrap=False, oob_score=False))

bagging_model.fit(X_train, y_train)
pasting_model.fit(X_train, y_train)

bagging_oob = bagging_model.named_steps["model"].oob_score_

bagging_summary = pd.DataFrame([
    {
        "Mô hình": "Bagging",
        "Bootstrap": True,
        "OOB Score": bagging_oob,
        "Test Accuracy": accuracy_score(y_test, bagging_model.predict(X_test)),
        "Test F1": f1_score(y_test, bagging_model.predict(X_test), zero_division=0)
    },
    {
        "Mô hình": "Pasting",
        "Bootstrap": False,
        "OOB Score": np.nan,
        "Test Accuracy": accuracy_score(y_test, pasting_model.predict(X_test)),
        "Test F1": f1_score(y_test, pasting_model.predict(X_test), zero_division=0)
    }
])

display(bagging_summary)

print("Nhận xét:")

```

```
print("Bagging có OOB score = {bagging_oob:.4f}.")
print("Bagging và Pasting đều làm giảm sự phụ thuộc vào một cây đơn lẻ bằng cách trung bình hóa nhiều cây.")
print("Nếu OOB score gần Test Accuracy, đánh giá nội bộ của Bagging phản ánh khá tốt hiệu quả trên dữ liệu chưa thấy.")
```

	Mô hình	Bootstrap	OOB Score	Test Accuracy	Test F1	
0	Bagging	True	0.832335	0.820628	0.740260	
1	Pasting	False	NaN	0.816143	0.738854	

Nhận xét:

Bagging có OOB score = 0.8323.

Bagging và Pasting đều làm giảm sự phụ thuộc vào một cây đơn lẻ bằng cách trung bình hóa nhiều cây.

Nếu OOB score gần Test Accuracy, đánh giá nội bộ của Bagging phản ánh khá tốt hiệu quả trên dữ liệu chưa thấy.

Các bước tiếp theo: [Tạo mã bảng bagging_summary](#) [New interactive sheet](#)

11. Random Forest và Extra Trees

Random Forest mở rộng Bagging bằng cách chọn ngẫu nhiên thêm tập feature khi tìm split. Extra Trees còn tăng ngẫu nhiên ở ngưỡng tách. Hai mô hình này thường giảm phương sai tốt hơn Decision Tree đơn lẻ, đổi lại khả năng diễn giải từng quyết định không còn trực tiếp như một cây duy nhất.

```
rf_model = make_pipeline(RandomForestClassifier(
    n_estimators=250,
    max_depth=None,
    min_samples_leaf=3,
    max_features="sqrt",
    oob_score=True,
    random_state=RANDOM_STATE,
    n_jobs=-1
))

extra_trees_model = make_pipeline(ExtraTreesClassifier(
    n_estimators=250,
    max_depth=None,
    min_samples_leaf=3,
    max_features="sqrt",
    random_state=RANDOM_STATE,
    n_jobs=-1
))

rf_model.fit(X_train, y_train)
extra_trees_model.fit(X_train, y_train)

rf_oob = rf_model.named_steps["model"].oob_score_

forest_summary = pd.DataFrame([
    {
        "Mô hình": "Random Forest",
        "OOB Score": rf_oob,
        "Test Accuracy": accuracy_score(y_test, rf_model.predict(X_test)),
        "Test F1": f1_score(y_test, rf_model.predict(X_test), zero_division=0)
    },
    {
        "Mô hình": "Extra Trees",
        "OOB Score": np.nan,
        "Test Accuracy": accuracy_score(y_test, extra_trees_model.predict(X_test)),
        "Test F1": f1_score(y_test, extra_trees_model.predict(X_test), zero_division=0)
    }
])

display(forest_summary)

print("Nhận xét:")
print("Random Forest lấy trung bình nhiều cây được huấn luyện trên các mẫu và feature khác nhau, nên thường ổn định")
print("Extra Trees tăng thêm ngẫu nhiên ở ngưỡng tách; điều này có thể giảm phương sai nhưng đôi khi tăng bias.")
print(f"Trong lần chạy này, Random Forest có OOB score = {rf_oob:.4f}.")
```

	Mô hình	00B Score	Test Accuracy	Test F1	
0	Random Forest	0.827844	0.820628	0.740260	
1	Extra Trees	NaN	0.816143	0.735484	

Nhận xét:

Random Forest lấy trung bình nhiều cây được huấn luyện trên các mẫu và feature khác nhau, nên thường ổn định hơn Deci Extra Trees tăng thêm ngẫu nhiên ở ngưỡng tách; điều này có thể giảm phương sai nhưng đôi khi tăng bias. Trong lần chạy này, Random Forest có 00B score = 0.8278.

Các bước tiếp theo: [Tạo mã bằng forest_summary](#) [New interactive sheet](#)

12. Boosting

Boosting học tuần tự: mô hình sau tập trung sửa phần mà mô hình trước dự đoán chưa tốt. Nhóm này thường mạnh trên dữ liệu bảng, nhưng nhạy với số vòng lặp, learning rate và độ sâu của cây. Vì vậy các tham số được giữ ở mức vừa phải để tránh mô hình quá bám tập train.

```
boosting_models = {
    "AdaBoost": make_pipeline(AdaBoostClassifier(
        n_estimators=160,
        learning_rate=0.05,
        random_state=RANDOM_STATE
    )),
    "Gradient Boosting": make_pipeline(GradientBoostingClassifier(
        n_estimators=160,
        learning_rate=0.05,
        max_depth=3,
        random_state=RANDOM_STATE
    )),
    "HistGradientBoosting": make_pipeline(HistGradientBoostingClassifier(
        learning_rate=0.05,
        max_iter=180,
        max_leaf_nodes=15,
        random_state=RANDOM_STATE
    ))
}

try:
    from xgboost import XGBClassifier

    boosting_models["XGBoost"] = make_pipeline(XGBClassifier(
        n_estimators=160,
        learning_rate=0.05,
        max_depth=3,
        subsample=0.9,
        colsample_bytree=0.9,
        eval_metric="logloss",
        random_state=RANDOM_STATE,
        n_jobs=-1
    ))
    xgb_status = "Có XGBoost trong môi trường, đã thêm vào so sánh."
except Exception:
    xgb_status = "Không import được XGBoost, bỏ qua mô hình này."

print(xgb_status)

boosting_rows = []
for name, model in boosting_models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    boosting_rows.append({
        "Mô hình": name,
        "Accuracy": accuracy_score(y_test, pred),
        "Precision": precision_score(y_test, pred, zero_division=0),
        "Recall": recall_score(y_test, pred, zero_division=0),
        "F1": f1_score(y_test, pred, zero_division=0)
    })

boosting_df = pd.DataFrame(boosting_rows).sort_values("F1", ascending=False)
display(boosting_df)

best_boosting = boosting_df.iloc[0]
```



```
print("Nhận xét: ")
print(f"Trong nhóm Boosting, {best_boosting['Mô hình']} có F1 cao nhất = {best_boosting['F1']:.4f}.")
print("Boosting không luôn thắng tuyệt đối; hiệu quả phụ thuộc vào dữ liệu, tham số và mức nhiễu của tập train.")
```

Có XGBoost trong môi trường, đã thêm vào so sánh.

	Mô hình	Accuracy	Precision	Recall	F1	
1	Gradient Boosting	0.820628	0.810811	0.697674	0.750000	
2	HistGradientBoosting	0.807175	0.786667	0.686047	0.732919	
3	XGBoost	0.793722	0.785714	0.639535	0.705128	
0	AdaBoost	0.757848	0.690476	0.674419	0.682353	

Nhận xét:

Trong nhóm Boosting, Gradient Boosting có F1 cao nhất = 0.7500.

Boosting không luôn thắng tuyệt đối; hiệu quả phụ thuộc vào dữ liệu, tham số và mức nhiễu của tập train.

Các bước tiếp theo:

[Tạo mã bằng boosting_df](#)

[New interactive sheet](#)

13. So sánh tổng hợp các mô hình

Tất cả mô hình được đánh giá trên cùng train/test split. Ngoài kết quả trên test, cross-validation 5 folds được dùng để quan sát độ ổn định.

(F1) được ưu tiên vì nó cân bằng Precision và Recall, còn (ROC-AUC) phản ánh khả năng xếp hạng xác suất trên nhiều ngưỡng khác nhau.

```
all_models = {
    "Baseline": baseline,
    **base_models,
    "Voting Hard": voting_hard,
    "Voting Soft": voting_soft,
    "Bagging": bagging_model,
    "Pasting": pasting_model,
    "Random Forest": rf_model,
    "Extra Trees": extra_trees_model,
    **boosting_models
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

def get_score_vector(model, X_data):
    if hasattr(model, "predict_proba"):
        try:
            return model.predict_proba(X_data)[: , 1]
        except Exception:
            pass
    if hasattr(model, "decision_function"):
        try:
            return model.decision_function(X_data)
        except Exception:
            pass
    return None

rows = []
fitted_models = {}

for name, model in all_models.items():
    start = time.time()
    model.fit(X_train, y_train)
    train_time = time.time() - start

    pred = model.predict(X_test)
    score = get_score_vector(model, X_test)

    row = {
        "Mô hình": name,
        "Accuracy": accuracy_score(y_test, pred),
        "Precision": precision_score(y_test, pred, zero_division=0),
        "Recall": recall_score(y_test, pred, zero_division=0),
        "F1": f1_score(y_test, pred, zero_division=0),
        "ROC-AUC": roc_auc_score(y_test, score) if score is not None else np.nan,
        "Train time (s)": train_time
    }

    try:
        cv_scores = cross_validate(
```

```

        model,
        X_train,
        y_train,
        cv=cv,
        scoring=["accuracy", "f1", "roc_auc"],
        n_jobs=-1
    )
    row["CV Accuracy mean"] = cv_scores["test_accuracy"].mean()
    row["CV F1 mean"] = cv_scores["test_f1"].mean()
    row["CV ROC-AUC mean"] = cv_scores["test_roc_auc"].mean()
    row["CV F1 std"] = cv_scores["test_f1"].std()
except Exception:
    row["CV Accuracy mean"] = np.nan
    row["CV F1 mean"] = np.nan
    row["CV ROC-AUC mean"] = np.nan
    row["CV F1 std"] = np.nan

rows.append(row)
fitted_models[name] = model

results_df = pd.DataFrame(rows).sort_values("F1", ascending=False)
display(results_df)

best_model_name = results_df.iloc[0]["Mô hình"]
best_model = fitted_models[best_model_name]
best_auc_name = results_df.dropna(subset=["ROC-AUC"]).sort_values("ROC-AUC", ascending=False).iloc[0]["Mô hình"]

print("Nhận xét:")
print(f"Mô hình tốt nhất theo Test F1 là {best_model_name} với F1 = {results_df.iloc[0]['F1']:.4f}.")
print(f"Mô hình có ROC-AUC cao nhất là {best_auc_name}.")
print("Nếu một mô hình có Accuracy cao nhưng F1 thấp, mô hình đó có thể đang dự đoán tốt lớp đa số hơn lớp số ít.")
print("Kết quả ensemble cần được đọc cùng CV mean và CV std, vì một điểm test cao chưa đủ chứng minh mô hình ổn định")

```

	Mô hình	Accuracy	Precision	Recall	F1	ROC-AUC	Train time (s)	CV Accuracy mean	CV F1 mean	CV ROC-AUC mean	CV F1 std
1	Logistic Regression	0.825112	0.783133	0.755814	0.769231	0.874767	0.091686	0.820312	0.755565	0.863757	0.026833
3	Decision Tree	0.820628	0.767442	0.767442	0.767442	0.851553	0.076302	0.815846	0.746670	0.848645	0.025189
5	Voting Hard	0.825112	0.830986	0.686047	0.751592	NaN	0.208022	NaN	NaN	NaN	NaN
6	Voting Soft	0.816143	0.784810	0.720930	0.751515	0.863987	3.419538	0.817304	0.745666	0.870327	0.027781
12	Gradient Boosting	0.820628	0.810811	0.697674	0.750000	0.861781	0.244817	0.823319	0.758122	0.878013	0.014536
7	Bagging	0.820628	0.838235	0.662791	0.740260	0.852699	0.371322	0.823319	0.753582	0.864423	0.018161
9	Random Forest	0.820628	0.838235	0.662791	0.740260	0.861526	0.527754	0.814342	0.741780	0.872082	0.021449
8	Pasting	0.816143	0.816901	0.674419	0.738854	0.851341	0.386043	0.830872	0.768400	0.863488	0.034848
10	Extra Trees	0.816143	0.826087	0.662791	0.735484	0.860635	0.283092	0.827797	0.759560	0.864725	0.024427
4	SVM RBF	0.807175	0.786667	0.686047	0.732919	0.850323	0.147988	0.833767	0.765211	0.848065	0.037380
13	HistGradientBoosting	0.807175	0.786667	0.686047	0.732919	0.821550	0.157906	0.814398	0.748700	0.867845	0.020539
14	XGBoost	0.793722	0.785714	0.639535	0.705128	0.855585	0.037907	0.824823	0.760380	0.879029	0.017727
2	KNN	0.780269	0.746667	0.651163	0.695652	0.847819	0.049521	0.805387	0.732505	0.855237	0.013939
11	AdaBoost	0.757848	0.690476	0.674419	0.682353	0.836106	0.245470	0.796386	0.733557	0.864670	0.029285
0	Baseline	0.614350	0.000000	0.000000	0.000000	0.500000	0.015709	0.616766	0.000000	0.500000	0.000000

Nhận xét:
Mô hình tốt nhất theo Test F1 là Logistic Regression với F1 = 0.7692.
Mô hình có ROC-AUC cao nhất là Logistic Regression.

Các bước tiếp theo: [Tạo mã bằng results_df](#) [New interactive sheet](#)

```

plot_df = results_df[["Mô hình", "Accuracy", "F1", "ROC-AUC"]].copy()
plot_df = plot_df.sort_values("F1", ascending=True)

fig, axes = plt.subplots(1, 2, figsize=(15, 7))

sns.barplot(data=plot_df, y="Mô hình", x="F1", ax=axes[0])
axes[0].set_title("So sánh F1 theo mô hình")
axes[0].set_xlim(0, 1)
axes[0].set_xlabel("F1")

```

```

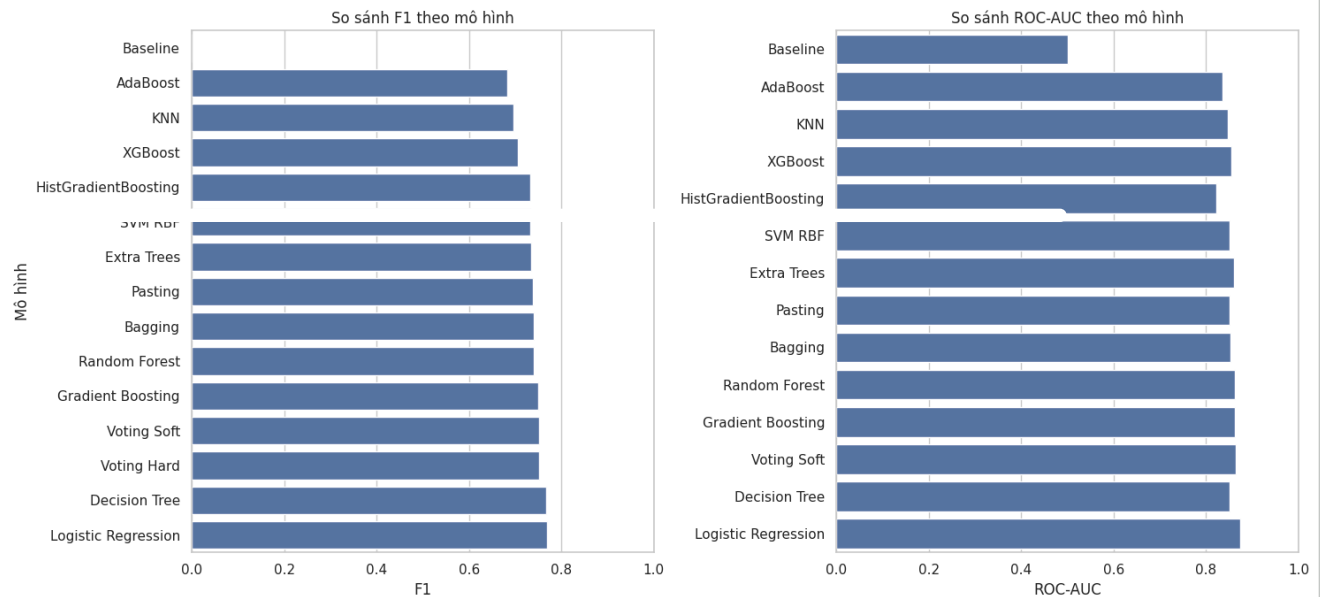
axes[0].set_ylabel("Mô hình")

auc_plot_df = plot_df.dropna(subset=["ROC-AUC"])
sns.barplot(data=auc_plot_df, y="Mô hình", x="ROC-AUC", ax=axes[1])
axes[1].set_title("So sánh ROC-AUC theo mô hình")
axes[1].set_xlim(0, 1)
axes[1].set_xlabel("ROC-AUC")
axes[1].set_ylabel("")

plt.tight_layout()
plt.show()

print("Nhận xét:")
print("F1 cho thấy chất lượng phân loại ở ngưỡng mặc định, còn ROC-AUC cho thấy khả năng xếp hạng xác suất trên nhiều ngưỡng")
print("Một số ensemble có ROC-AUC khá cao nhưng F1 không đứng đầu, nghĩa là xác suất có tín hiệu tốt nhưng ngưỡng 0.5 chưa c

```



Nhận xét:
F1 cho thấy chất lượng phân loại ở ngưỡng mặc định, còn ROC-AUC cho thấy khả năng xếp hạng xác suất trên nhiều threshold.
Một số ensemble có ROC-AUC khá cao nhưng F1 không đứng đầu, nghĩa là xác suất có tín hiệu tốt nhưng ngưỡng 0.5 chưa c

14. Confusion Matrix của mô hình tốt nhất theo F1

Confusion Matrix cho biết lỗi cụ thể của mô hình. Với Titanic, false positive là dự đoán sống sót nhưng thực tế không sống sót. False negative là dự đoán không sống sót nhưng thực tế sống sót. Hai loại lỗi này giúp nhìn rõ hơn so với chỉ độc Accuracy.

```

best_pred = best_model.predict(X_test)
cm_best = confusion_matrix(y_test, best_pred)

plt.figure(figsize=(6, 5))
sns.heatmap(
    cm_best,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Không sống sót", "Sống sót"],
    yticklabels=["Không sống sót", "Sống sót"]
)
plt.xlabel("Dự đoán")

```

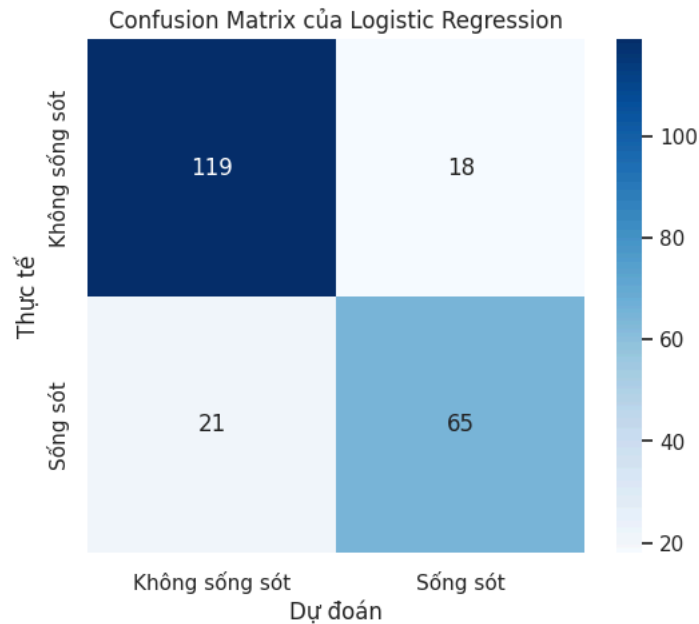
```

plt.ylabel("Thực tế")
plt.title(f"Confusion Matrix của {best_model_name}")
plt.show()

print("Classification report:")
print(classification_report(y_test, best_pred, target_names=["Không sống sót", "Sống sót"], zero_division=0))

fp = cm_best[0, 1]
fn = cm_best[1, 0]
print("Nhận xét:")
print(f"{best_model_name} dự đoán sai {cm_best.sum() - np.trace(cm_best)} mẫu trên tập test.")
print(f"False Positive = {fp}, False Negative = {fn}.")
if fn > fp:
    print("Mô hình bỏ sót hành khách sống sót nhiều hơn số trường hợp dự đoán sống sót sai.")
elif fp > fn:
    print("Mô hình dự đoán sống sót sai nhiều hơn số trường hợp bỏ sót hành khách sống sót.")
else:
    print("Hai loại lỗi đang xuất hiện ở mức tương đương.")

```



```

Classification report:
              precision    recall  f1-score   support

Không sống sót      0.85      0.87      0.86       137
Sống sót            0.78      0.76      0.77        86

   accuracy              0.83       223
  macro avg              0.82       223
 weighted avg              0.82       223

```

Nhận xét:
 Logistic Regression dự đoán sai 39 mẫu trên tập test.
 False Positive = 18, False Negative = 21.
 Mô hình bỏ sót hành khách sống sót nhiều hơn số trường hợp dự đoán sống sót sai.

15. ROC Curve

ROC Curve thể hiện khả năng tách hai lớp khi thay đổi threshold. Đường cong càng gần góc trên bên trái thì mô hình càng xếp điểm lớp sống sót cao hơn lớp không sống sót một cách ổn định.

```

top_auc_df = results_df.dropna(subset=["ROC-AUC"]).sort_values("ROC-AUC", ascending=False).head(5)

plt.figure(figsize=(8, 6))
for name in top_auc_df["Mô hình"]:
    model = fitted_models[name]
    score = get_score_vector(model, X_test)
    if score is None:
        continue
    fpr, tpr, _ = roc_curve(y_test, score)
    auc = roc_auc_score(y_test, score)

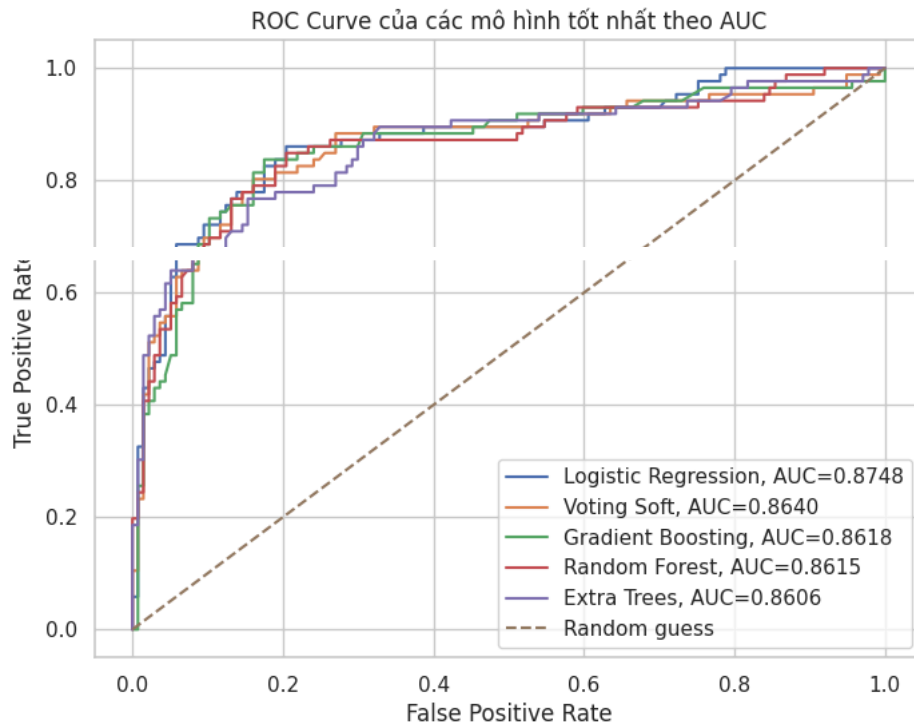
```

```
plt.plot(fpr, tpr, label=f"{name}, AUC={auc:.4f}")

plt.plot([0, 1], [0, 1], linestyle="--", label="Random guess")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve của các mô hình tốt nhất theo AUC")
plt.legend()
plt.show()

display(top_auc_df[["Mô hình", "ROC-AUC", "F1", "Accuracy"]])

print("Nhận xét:")
print("Các đường ROC nằm cao hơn đường chéo ngẫu nhiên, cho thấy mô hình có khả năng phân biệt hai lớp.")
print("Trong kết quả này, nhóm mô hình đứng đầu theo ROC-AUC có điểm khá gần nhau; chênh lệch nhỏ nên cần đọc thêm F
```



	Mô hình	ROC-AUC	F1	Accuracy	
1	Logistic Regression	0.874767	0.769231	0.825112	
6	Voting Soft	0.863987	0.751515	0.816143	
12	Gradient Boosting	0.861781	0.750000	0.820628	
9	Random Forest	0.861526	0.740260	0.820628	
10	Extra Trees	0.860635	0.735484	0.816143	

Nhận xét:

Các đường ROC nằm cao hơn đường chéo ngẫu nhiên, cho thấy mô hình có khả năng phân biệt hai lớp.

Trong kết quả này, nhóm mô hình đứng đầu theo ROC-AUC có điểm khá gần nhau; chênh lệch nhỏ nên cần đọc thêm F1 và cor

16. OOB score

OOB score chỉ có ở mô hình dùng bootstrap. Mỗi cây được huấn luyện trên một mẫu bootstrap và được kiểm tra trên phần mẫu không xuất hiện trong bootstrap đó. Vì vậy OOB score có thể xem như một phép đánh giá nội bộ, đặc biệt hữu ích với Bagging và Random Forest.

```
oob_rows = []

if hasattr(bagging_model.named_steps["model"], "oob_score_"):
    oob_rows.append({
        "Mô hình": "Bagging",
        "OOB Score": bagging_model.named_steps["model"].oob_score_,
        "Test Accuracy": accuracy_score(y_test, bagging_model.predict(X_test))
    })

if hasattr(rf_model.named_steps["model"], "oob_score_"):
    oob_rows.append({
```

```

        "Mô hình": "Random Forest",
        "00B Score": rf_model.named_steps["model"].oob_score_,
        "Test Accuracy": accuracy_score(y_test, rf_model.predict(X_test))
    })

oob_df = pd.DataFrame(oob_rows)
display(oob_df)

print("Nhận xét:")
if len(oob_df) > 0:
    oob_df["Độ lệch tuyệt đối"] = (oob_df["00B Score"] - oob_df["Test Accuracy"]).abs()
    gap = oob_df["Độ lệch tuyệt đối"].max()
    print(f"Độ lệch lớn nhất giữa 00B Score và Test Accuracy là {gap:.4f}.")
    print("Khoảng cách nhỏ cho thấy 00B score phản ánh tương đối tốt hiệu quả trên dữ liệu chưa thấy.")
else:
    print("Không có mô hình nào trong danh sách hiện tại hỗ trợ 00B score.")

```

	Mô hình	00B Score	Test Accuracy	
0	Bagging	0.832335	0.820628	
1	Random Forest	0.827844	0.820628	

Nhận xét:

Độ lệch lớn nhất giữa 00B Score và Test Accuracy là 0.0117.

Khoảng cách nhỏ cho thấy 00B score phản ánh tương đối tốt hiệu quả trên dữ liệu chưa thấy.

Các bước tiếp theo:

[Tạo mã bằng oob_df](#)

[New interactive sheet](#)

17. Feature importance của Random Forest

Feature importance cho biết biến nào được Random Forest sử dụng nhiều để giảm độ lẫn lộn trong các cây. Chỉ số này không chứng minh quan hệ nhân quả, nhưng giúp liên hệ kết quả mô hình với phân tích EDA ban đầu.

```

def clean_feature_name(name):
    return str(name).replace("num__", "").replace("cat__", "")

rf_fitted = fitted_models["Random Forest"]
preprocessor_fitted = rf_fitted.named_steps["preprocess"]
rf_classifier = rf_fitted.named_steps["model"]

try:
    feature_names = preprocessor_fitted.get_feature_names_out()
except Exception:
    feature_names = [f"feature_{i}" for i in range(len(rf_classifier.feature_importances_))]

importance_df = pd.DataFrame({
    "Feature": [clean_feature_name(f) for f in feature_names],
    "Importance": rf_classifier.feature_importances_
}).sort_values("Importance", ascending=False)

display(importance_df.head(15))

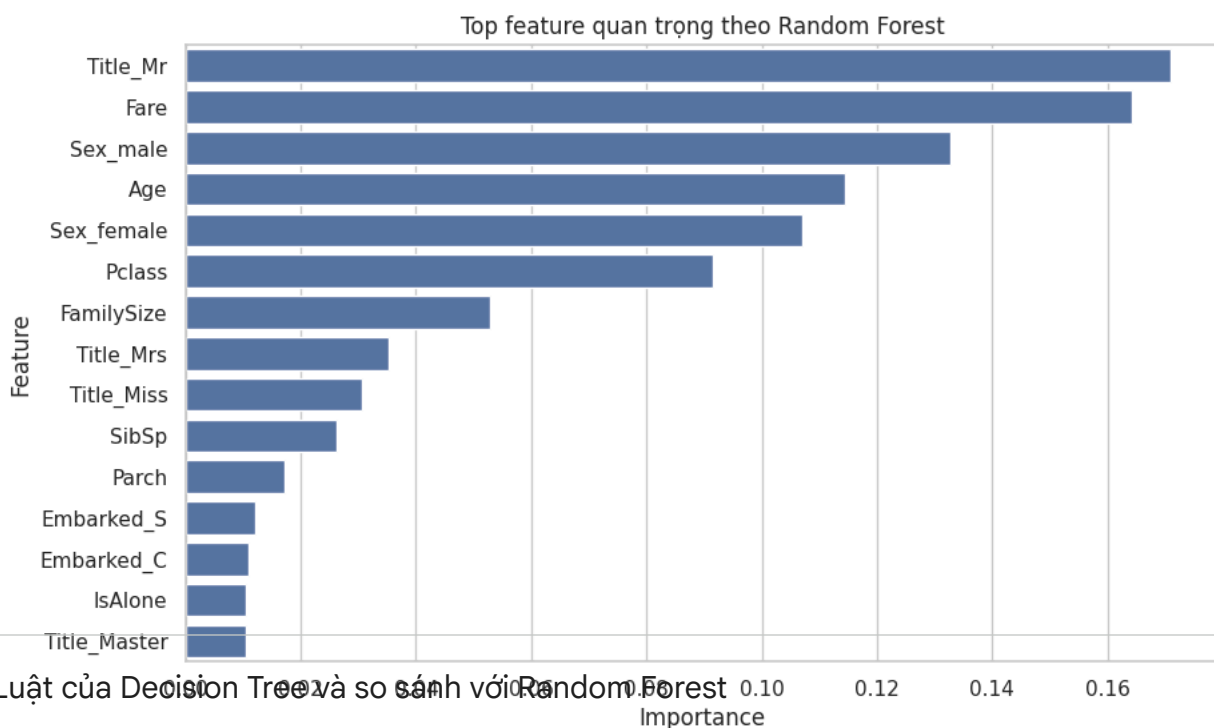
plt.figure(figsize=(10, 6))
sns.barplot(data=importance_df.head(15), x="Importance", y="Feature")
plt.title("Top feature quan trọng theo Random Forest")
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.show()

top_feature = importance_df.iloc[0]

print("Nhận xét:")
print(f"Feature quan trọng nhất là {top_feature['Feature']} với importance = {top_feature['Importance']:.4f}.")
print("Các feature nổi bật thường liên quan đến Title, Sex, Fare, Age và Pclass, phù hợp với phân EDA về giới tính,")
print("Feature importance cho biết mức đóng góp trong mô hình, không nên diễn giải trực tiếp thành nguyên nhân xã hội")

```

	Feature	Importance	
14	Title_Mr	0.170881	
4	Fare	0.164211	
8	Sex_male	0.132809	
1	Age	0.114380	
7	Sex_female	0.107024	
0	Pclass	0.091609	
5	FamilySize	0.052959	
15	Title_Mrs	0.035209	
13	Title_Miss	0.030588	
2	SibSp	0.026180	
3	Parch	0.017213	
11	Embarked_S	0.012171	
9	Embarked_C	0.010986	
6	IsAlone	0.010530	
12	Title_Master	0.010464	



18. Luật của Decision Tree và so sánh với Random Forest

Decision Tree đơn lẻ có ưu điểm là dễ chuyển thành luật. Tuy nhiên một cây duy nhất thường nhạy với cách chia dữ liệu. Random Forest giảm điểm yếu này bằng cách trung bình hóa nhiều cây khác nhau. Phần này chỉ lấy một số luật là của Decision Tree để đọc cấu trúc. Các feature nổi bật thường liên quan đến Title, Sex, Fare, Age và Pclass, phù hợp với phân EDA về giới tính, hạng và quyết định sau đó so sánh metric với Random Forest trong mô hình, không nên diễn giải trực tiếp thành nguyên nhân xã hội hay

```
tree_model = fitted_models["Decision Tree"]
tree_classifier = tree_model.named_steps["model"]

try:
    tree_feature_names = [clean_feature_name(f) for f in tree_model.named_steps["preprocess"].get_feature_names_out()]
except Exception:
    tree_feature_names = [f"feature_{i}" for i in range(tree_classifier.n_features_in_)]

def extract_leaf_rules(decision_tree, feature_names, max_rules=12):
```